Angular. Instead, it's a standard HTTP call sent from the browser, like the following:

```
login(): void {
  window.open('/oauth2/authorization/keycloak', '_self');
}
```

I'd like to stress that the login call is not an AJAX request sent from the Angular `HttpClient`. Instead, it instructs the browser to call the login URL. Spring Security exposes an `/oauth2/authorization/{registrationId}` endpoint that you can use to start the authentication flow based on OAuth2/OIDC. Since the client registration identifier for Edge Service is `keycloak`, the login endpoint will be `/oauth2/authorization/ keycloak`.

To make that possible, we need to define a custom `AuthenticationEntryPoint` to instruct Spring Security to reply with an HTTP `401` status when an unauthenticated request is received for a protected resource. The framework already provides an `HttpStatusServerEntryPoint` implementation that perfectly suits this scenario, since it lets you specify which HTTP status to return when a user is required to authenticate.

Listing 11.14 Returning 401 when the user is not authenticated

```
@EnableWebFluxSecurity
public class SecurityConfig {
  ...

  @Bean
  SecurityWebFilterChain springSecurityFilterChain(
    ServerHttpSecurity http,
    ReactiveClientRegistrationRepository clientRegistrationRepository
  ) {
    return http
      .authorizeExchange(exchange -> exchange.anyExchange().authenticated())
      .exceptionHandling(exceptionHandling ->
```